기구학 정리본

▼ 정기구학

왜 정기구학을 배우는가?

"로봇의 관절값이 주어졌을 때, 로봇의 각 부분과 끝점이 어디에 있는지 계산하는 방법"

예를 들어,

- 첫 번째 관절이 30도 회전
- 두 번째 관절이 45도 회전

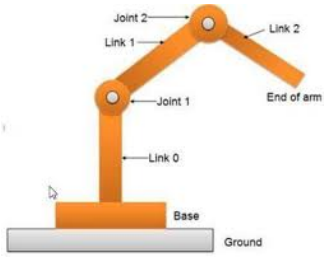
라고 한다면 이 로봇의 끝 부분은 어디인가? 를 구하는 것이 정기구학이다.

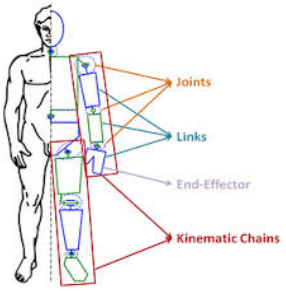
가장 많이 나오는 단어 4가지

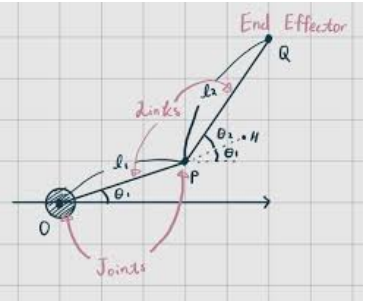
항목	핵심 개념	비유 (쉬운 설명)	실무 포인트
Link	로봇의 **뼈대**	움직임을 **당하는** 단단한 조각	무게와 길이를 계산할 때 중요
Joint	로봇의 **관절**	링크와 링크의 **연결점**	회전(Revolute) vs 직선(Prismatic)
Frame	**기준 좌표계**	위치를 말하기 위한 **기준점**	"어디 기준인가?"가 가장 중요 (TF)
End Effector	로봇의 **끝점**	로봇의 **손가락/작업 도구**	정기구학의 최종 목적지 (TCP)

cf) tf : transform

예시 이미지







URDF - 플레밍의 오른손 법칙

[좌표 및 색깔]

- x축 (R) - 검지
- y축 (G) - 중지
- z축 (B) - 엄지

[회전 방향]

- **Roll** : x축 중심 회전 (기울기)
- **Pitch** : y축 중심 회전 (끄덕끄덕)
- **Yaw** : z축 중심 회전 (도리도리)

회전 후 이동 vs 이동 후 회전

```
import numpy as np

# 점
p = np.array([1, 0])

# 회전 (90도)
theta = np.deg2rad(90)
R = np.array([
    [np.cos(theta), -np.sin(theta)],
    [np.sin(theta),  np.cos(theta)]
])

# 이동
t = np.array([1, 1])

# Case 1: 회전 후 이동
p1 = R @ p + t
```

```
# Case 2: 이동 후 회전
p2 = R @ (p + t)

print("회전 후 이동:", p1)
print("이동 후 회전:", p2)
```

👉 p1과 p2의 값은 다르게 나온다.

▼ 역기구학

왜 역기구학을 배우는가?

정기구학과 비교하여 설명하면, 아래와 같다.

구분	입력 (Input)	출력 (Output)	핵심 질문 (Question)
정기구학 (FK)	관절 각도	손끝 위치	"관절을 이렇게 꺾으면 **손이 어디로 갈까?**"
역기구학 (IK)	**손끝 목표 위치**	**관절 각도**	"**저 물체를 잡으려면** 관절을 어떻게 꺾어야 할까?"

BUT , 정기구학은 답이 하나지만, 역기구학은 답이 여러 개(다중 해)일 수도 있고,
아예 팔이 닿지 않아 **답이 없을 수도** 있다.

---

x,y의 각도를 구해주는 코드

```
import math

x = 1
y = 1

theta = math.atan2(y, x)  # y,x 순서 기억할 것.
# 회전 각도 (도 → 라디안)
deg = np.rad2deg(theta) # theta = 라디안
```

```
print(deg)  # 45도 출력

# *** atan 말고 atan2를 사용해야 함.  (atan = 방향성 X / ata
n2 = 방향성 O ) ***
```

2링크 로봇 팔 역기구학 구현 코드

```
import math

# L1,L2 : 링크1,2의 길이 / x,y : 목표 끝단 좌표
def inverse_kinematics(x, y, L1, L2):
    # 거리
    d = math.sqrt(x**2 + y**2)

    # 도달 가능 체크
    if d > (L1 + L2) or d < abs(L1 - L2):
        return None

    # theta2
    cos_theta2 = (x**2 + y**2 - L1**2 - L2**2) / (2 * L1
* L2)
    cos_theta2 = max(-1, min(1, cos_theta2))

    theta2 = math.acos(cos_theta2)

    # theta1
    alpha = math.atan2(y, x)
    beta = math.atan2(
        L2 * math.sin(theta2),
        L1 + L2 * math.cos(theta2)
    )

    theta1 = alpha - beta

    return theta1, theta2

# === [실제값 대입 구간] ===
```

```python
L1 = 5 ; L2 = 3
x = 6 ; y = 2

result = inverse_kinematics(x, y, L1, L2)

if result:
    t1, t2 = result
    print("theta1:", math.degrees(t1))
    print("theta2:", math.degrees(t2))
else:
    print("도달 불가")
```